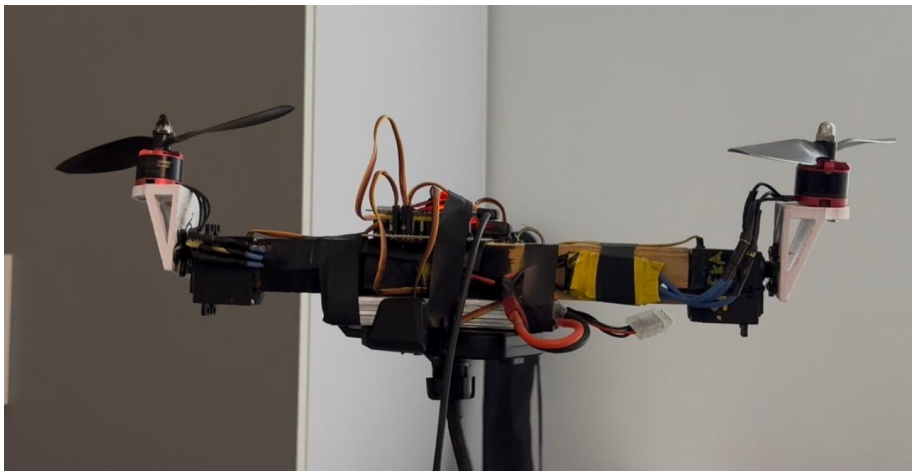
Attitude Control of a Tilt-Rotor Bicopter on an ESP32-S3

Design, implementation and experimental evaluation of a flight controller



Project report

Mohammed Abdelsabour

November 2026

Source code, data and figures:
https://github.com/mohammedabdelsaboor/BI-Copter-Flight-controller

Abstract

A bicopter is a multirotor with only two rotors. Because two rotors cannot steer by speed alone, each is mounted on a servo that tilts it forward and back: roll is controlled by differential thrust, while pitch and yaw are controlled by tilting the rotors. This report describes the design, implementation and experimental evaluation of a flight controller for such an aircraft, running on an ESP32-S3 microcontroller with an MPU6050 inertial sensor, two brushless motors and two MG996R servos.

Attitude is estimated with a Mahony complementary filter on a quaternion, extended with a smooth accelerometer weighting and an integral term that learns the gyroscope bias in flight. Each axis is stabilised by a cascaded controller, an outer proportional angle loop feeding an inner PID rate loop, executed at 500 Hz. Safety logic covers arming, radio failsafe, crash detection, take-off detection and a hardware watchdog. The firmware is organised as small, independent C++ libraries and was tuned against flight data logged on the real aircraft.

The experiments show a clear asymmetry between the axes. The roll axis, actuated by the motors, tracked its commanded rate with an RMS error of 16 deg/s, whereas the pitch axis, actuated by the servos, showed an error of 80 deg/s and oscillated by up to 25 deg. The analysis attributes this mainly to the mechanics of the tilt axis: the slow response and free play of the servos, the short moment arm between the rotors and the centre of gravity, and a centre of gravity located behind the rotor axis. Firmware measures, in particular take-off detection and a simplified pitch controller, removed controller-induced contributions; the remaining limitation is mechanical.

Keywords: bicopter, tilt-rotor, attitude estimation, Mahony filter, cascaded PID control, ESP32-S3, embedded flight control

Contents

1 Introduction . . . 1

2 Background and Related Work . . . 2

2.1 Bicopters and tilt-rotor control . . . 2

2.2 Attitude estimation . . . 2

2.3 Cascaded attitude control . . . 2

3 System Description . . . 3

3.1 Airframe and tilt mechanism . . . 3

3.2 Hardware components . . . 4

3.3 Coordinate frame and sign conventions . . . 5

4 Modelling and Control Design . . . 6

4.1 Actuation and mixing . . . 6

4.2 Attitude estimation . . . 6

4.3 Cascaded attitude control . . . 6

4.4 Filtering . . . 7

4.5 Take-off detection and safety . . . 7

5 Implementation . . . 8

5.1 Software architecture . . . 8

5.2 Real-time loop . . . 9

5.3 Development and test tools . . . 9

6 Experimental Results . . . 10

6.1 Test method . . . 10

6.2 Roll and pitch behaviour . . . 10

6.3 Attitude estimation . . . 10

6.4 Servo response . . . 11

6.5 Recorded flight . . . 11

7 Discussion . . . 15

7.1 Limitations . . . 15

8 Conclusions . . . 16

References . . . 17

Appendix A Hardware Wiring . . . 18

Appendix B Setup and Tuning Procedure 19

Appendix C Console Commands 20

Appendix D Problems Encountered During Testing 21

List of Figures

Figure 1. Schematic of the airframe . . . 3

Figure 2. The bicopter built in this project . . . 4

Figure 3. Cascaded attitude control for one axis . . . 7

Figure 4. Data flow through the libraries in one pass of the control loop . . . 8

Figure 5. Measured (a) roll and pitch angle and (b) pitch servo command during a throttle sweep at 25 % throttle (console log, earlier controller version) . . . 10

Figure 6. Simulated pitch estimation error of the complementary filter and the Mahony filter

Figure 7. Simulated servo step response with and without a 40 ms phase lead (model, not a measurement) . . . 11

Figure 8. Recorded flight: (a) roll angle; (b) pitch angle and its hover-trim target . . . 12

Figure 9. Commanded and measured angular rate for (a) roll and (b) pitch, 5 s of the recorded flight . . . 13

Figure 10. Recorded flight: (a) ESC pulses; (b) servo pulses . . . 13

Figure 11. RMS rate tracking error per axis over the airborne part of the recorded flight 14

List of Tables

Table 1. Hardware components . . . 5

Table 2. Safety functions of the flight controller . . . 7

Table 3. Firmware libraries . . . 9

Table 4. Statistics of the recorded flight (airborne part, throttle above 30 %) . . . 12

Table 5. Pin assignment of the ESP32-S3 PowerFeather . . . 18

Table 6. USB console commands . . . 20

Table 7. Problems encountered, their causes and the corrective measures . . . 21

1 Introduction

Multirotors with four or more rotors control all three rotational axes by varying rotor speeds. A bicopter, with only two rotors, has enough thrust to fly but not enough independent speed changes to steer. The usual solution is to tilt the rotors: tilting both rotors forward lowers the nose, tilting them in opposite directions yaws the aircraft, and roll is still produced by running one rotor faster than the other [5]. The resulting airframe is compact and efficient, but its pitch and yaw axes depend on servos whose response is an order of magnitude slower than that of an electronic speed controller.

This project develops a flight controller for a home-built tilt-rotor bicopter. Its objectives are:

- a controller that is safe to test with and small enough to be read and modified without a large autopilot code base;
- an analysis, based on logged flight data, of where the instability on the servo-actuated axes comes from;
- firmware measures that reduce that instability, and an honest separation of what remains a mechanical limitation.

Chapter 2 reviews related work. Chapter 3 describes the aircraft and its hardware, and Chapter 4 the estimation and control design. Chapter 5 covers the implementation, Chapter 6 the experimental results, and Chapters 7 and 8 discuss the findings and conclude. The appendices contain the wiring, the step-by-step setup and tuning procedure, the console reference and the problems encountered during testing.

2 Background and Related Work

2.1 Bicopters and tilt-rotor control

Tilt-rotor bicopters have been studied as an efficient alternative to quadrotors for indoor flight. Qin et al. [5] presented Gemini, a compact bi-copter whose two rotors are tilted by servos, and showed that the configuration can fly efficiently but places high demands on the tilt actuators. In the open source domain, ArduPilot supports the bicopter as a frame class with tilt servos [6], while hobby flight controllers such as PlainFlightController [7], dRehmFlight [8], ESP-FC [9] and ESP-Drone [10] provide reference implementations of multirotor control on small microcontrollers; most of them target quadrotors.

2.2 Attitude estimation

Low-cost inertial sensors combine a gyroscope, which measures angular rate precisely but drifts, with an accelerometer, which measures the direction of gravity but is disturbed by vibration and by the aircraft's own accelerations. Complementary filters fuse the two by trusting the gyroscope at high frequencies and the accelerometer at low frequencies. Mahony et al. [1] formulated this as a nonlinear complementary filter on the rotation group with proportional and integral correction, and Madgwick [2] proposed a gradient-descent variant. The filter used here follows Mahony's formulation.

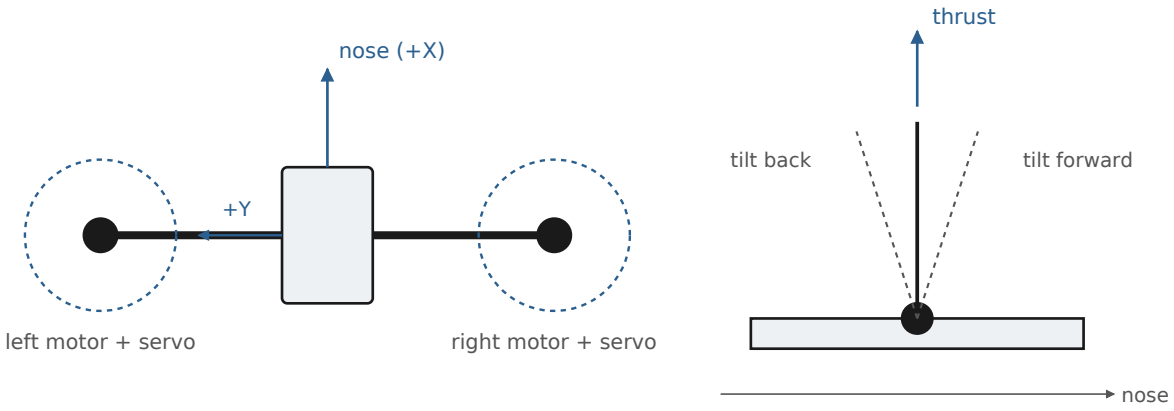
2.3 Cascaded attitude control

Multirotor attitude control is commonly implemented as a cascade of an outer angle loop and an inner angular-rate loop [3], [4]. The inner loop rejects disturbances quickly using the gyroscope, while the outer loop only has to define the desired attitude. Practical flight controllers add derivative filtering, integrator anti-windup and gain scheduling with throttle; Betaflight's documentation [11] summarises these techniques for small multirotors.

3 System Description

3.1 Airframe and tilt mechanism

The aircraft has a single arm with a rotor at each end (Figure 1). Each rotor is driven by a brushless motor sitting on a 3D-printed mount, which is turned about the arm by an MG996R servo. The flight controller, the inertial sensor and the battery are mounted at the centre of the arm. Figure 2 shows the aircraft built for this project.



(a) Top view (b) Side view of one tilt unit

Figure 1. Schematic of the airframe. (a) Top view with the body axes. (b) Side view of one tilt unit: the servo tilts the rotor forward or back, which changes the direction of its thrust.

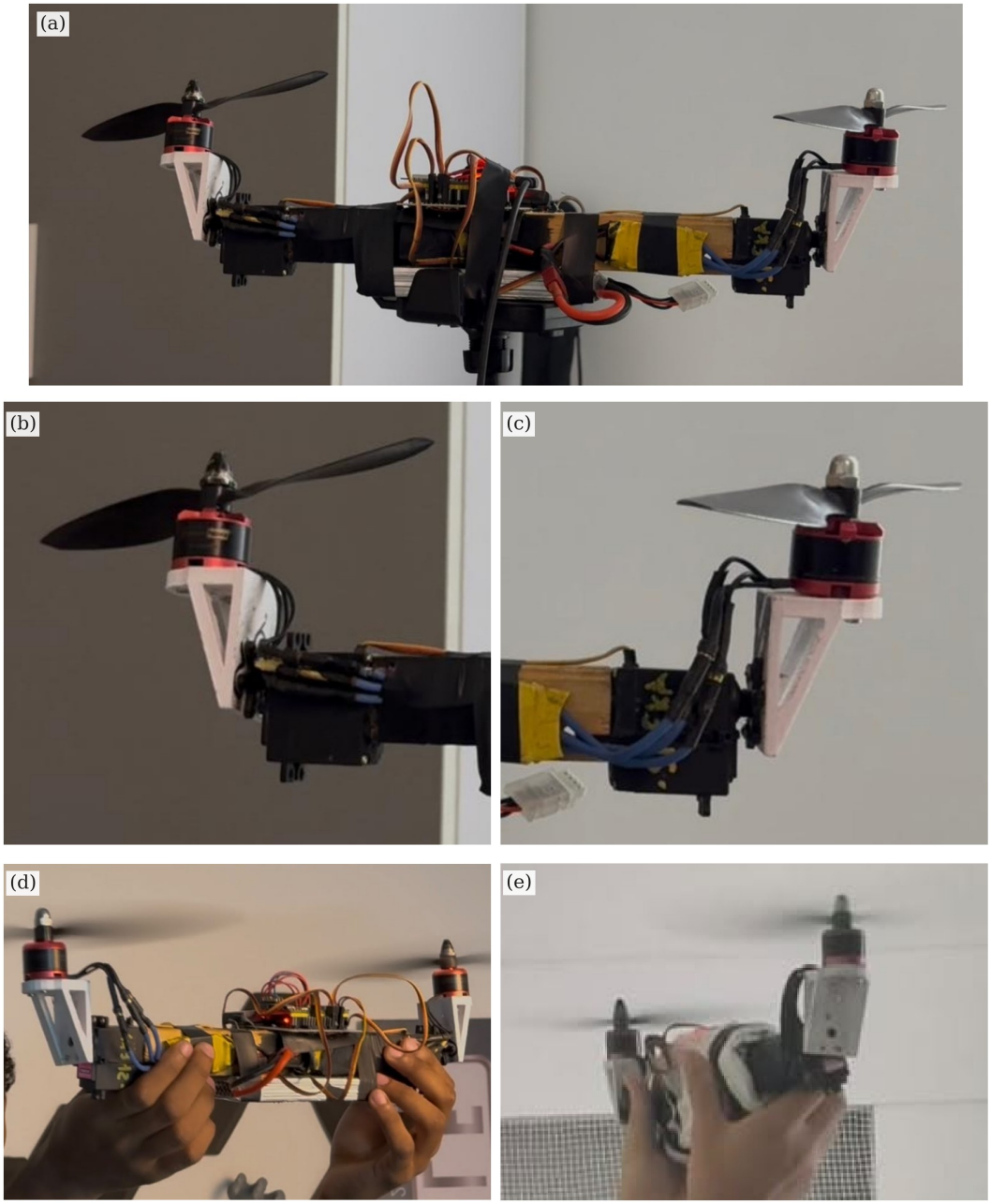


Figure 2. The bicopter built in this project. (a) Complete aircraft on its test stand. (b) Left and (c) right tilt unit: brushless motor on a printed mount, turned by an MG996R servo. (d) Aircraft held by hand with both motors running. (e) Aircraft held up during a flight test, seen from below.

3.2 Hardware components

The components are listed in Table 1. The servos are supplied by a separate regulator so that their current peaks do not disturb the flight controller.

Table 1. Hardware components.

Function	Component	Notes
Flight controller	ESP32-S3 PowerFeather	240 MHz dual core, native USB, WiFi
Inertial sensor	MPU6050 (GY-521 board)	3-axis gyroscope and accelerometer, I2C
Radio link	FlySky FS-i6 with FS-iA6B	four channels, one PWM signal each
Tilt actuators	2 × MG996R servo	about 0.17 s per 60 deg, updated at 100 Hz
Propulsion	2 × brushless motor with ESC	PWM at 400 Hz
Servo supply	5-6 V UBEC	separate from the controller supply

3.3 Coordinate frame and sign conventions

A single body frame is used throughout: X points to the nose, Y to the left rotor and Z upwards. Positive roll φ lowers the right side, positive pitch θ lowers the nose and positive yaw ψ turns the nose to the left. The sensor axes are mapped onto this frame by a rotation set in the configuration; the build rejects mappings that are mirror images, which is the most frequent mounting error.

4 Modelling and Control Design

4.1 Actuation and mixing

Each actuator serves more than one axis. With the collective thrust command T and the controller outputs u_φ, u_θ and u_ψ for roll, pitch and yaw, the motor commands m and the servo tilt commands s are

$$m_\mathrm{L} = T + u_\varphi, \quad m_\mathrm{R} = T - u_\varphi \tag{1}$$

$$s_\mathrm{L} = u_\theta - u_\psi, \quad s_\mathrm{R} = u_\theta + u_\psi \tag{2}$$

When a motor command would leave its range, both motor commands are shifted together so that the roll difference is preserved. Propeller thrust grows approximately with the square of the command, so the thrust demand τ is converted into an actuator command a using the thrust curve

$$\tau = (1 - e)a + e\,a^2 \;\Rightarrow\; a = [e - 1 + \sqrt{((1 - e)^2 + 4e\tau)}] \,/\, 2e \tag{3}$$

with $e = 0.65$, the same model as ArduPilot's thrust exponent. A roll command therefore produces the same torque at any throttle.

4.2 Attitude estimation

On a bicopter the accelerometer measures mainly the direction of the rotor thrust, which changes with every servo correction, so it can only be trusted on long time scales. The estimator is a Mahony filter [1] on the unit quaternion q. Let $\mathbf{a}_\mathrm{n}$ be the normalised, low-pass filtered accelerometer vector (second-order Butterworth, 10 Hz) and $\mathbf{v}$ the direction of gravity predicted by q. The correction is

$$\mathbf{e} = \mathbf{a}_\mathrm{n} \times \mathbf{v} \tag{4}$$

$$\mathbf{b} \leftarrow \mathbf{b} + k_\mathrm{i}\, w\, \mathbf{e}\, \Delta t \tag{5}$$

$$\boldsymbol{\omega}' = \boldsymbol{\omega} + k_\mathrm{p}\, w\, \mathbf{e} + \mathbf{b}, \quad \dot{q} = \tfrac{1}{2}\, q \otimes (0, \boldsymbol{\omega}') \tag{6}$$

with $k_\mathrm{p} = 1/3$ s^{-1} and $k_\mathrm{i} = k_\mathrm{p}/15$ s. The integral state $\mathbf{b}$ converges to the gyroscope bias, so the slow correction leaves no steady angle error. The accelerometer weight w falls linearly from 1 at exactly 1 g to 0 at 0.8 g and 1.2 g, because outside that range the measurement contains the aircraft's own acceleration:

$$w = \max(0,\, 1 - |\, \|\mathbf{a}\| - 1 \,| \,/\, 0.2) \tag{7}$$

4.3 Cascaded attitude control

Each axis uses two loops in series (Figure 3). The outer loop converts the angle error into a desired angular rate, and the inner loop drives the actuators so that the measured rate ω follows it:

$$\omega^* = K_\theta\, (\theta^* - \theta) \tag{8}$$

$$u = K_\mathrm{p}\, e_\omega + K_\mathrm{i} \int e_\omega \,\mathrm{d}t - K_\mathrm{d}\, \mathrm{d}\omega/\mathrm{d}t, \quad e_\omega = \omega^* - \omega \tag{9}$$

The derivative acts on the measurement rather than the error, so fast stick movements do not cause a kick, and the integrator stops while the output is saturated.

With the yaw stick centred, an outer heading loop of the same form holds the current heading.

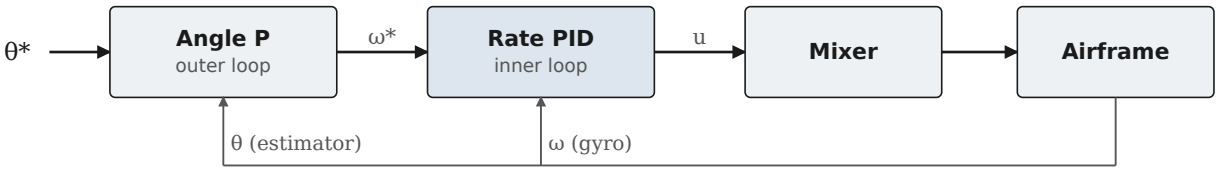


Figure 3. Cascaded attitude control for one axis. The outer loop uses the estimated angle θ, the inner loop the gyroscope rate ω.

4.4 Filtering

The sensor's internal low-pass filter is set to 44 Hz. In addition the gyroscope signal is filtered at 80 Hz for roll, 40 Hz for pitch and 25 Hz for yaw, and the servo commands at 30 Hz. Every filter adds phase lag, so the servo axes use the lightest filtering that still prevents vibration from making the servos chatter in their gear play.

4.5 Take-off detection and safety

On the ground the airframe cannot respond to corrections, and an active integrator accumulates correction against the ground. In testing this tilted the servos by 20-25 deg before lift-off and made the aircraft lunge forward as it left the ground. The integrators, heading hold and roll actuation are therefore enabled only when the throttle reaches a lift-off value (35 %), and disabled again only near idle. The remaining safety functions are summarised in Table 2.

Table 2. Safety functions of the flight controller.

Situation	Response
Power-up	ESC outputs set to 1000 µs (motor off) before any other initialisation.
Arming request	Refused unless the radio is valid, the throttle is low, the aircraft is within 25 deg of level, the gyroscope is calibrated and the stick centres are learned.
Radio link lost	Motors stop; re-arming requires the link to return with the throttle low.
Crash	Tilted beyond 60 deg for 0.5 s: disarm.
Idle while armed	15 s at idle: disarm.
Sensor failure	25 consecutive failed reads (50 ms): motors stop until reboot.
Software stall	300 ms watchdog resets the processor; all outputs go low.

5 Implementation

5.1 Software architecture

The firmware is a PlatformIO project for the Arduino framework on the ESP32-S3. The main sketch only connects the components and runs the control loop; every function lives in a small library with one header and one source file. No library reads the project configuration directly: each receives a settings structure from the sketch, which keeps the libraries independent and allows the estimator, the filters and the PID to be compiled and tested on a PC. Figure 4 shows the data flow and Table 3 the role of each library.

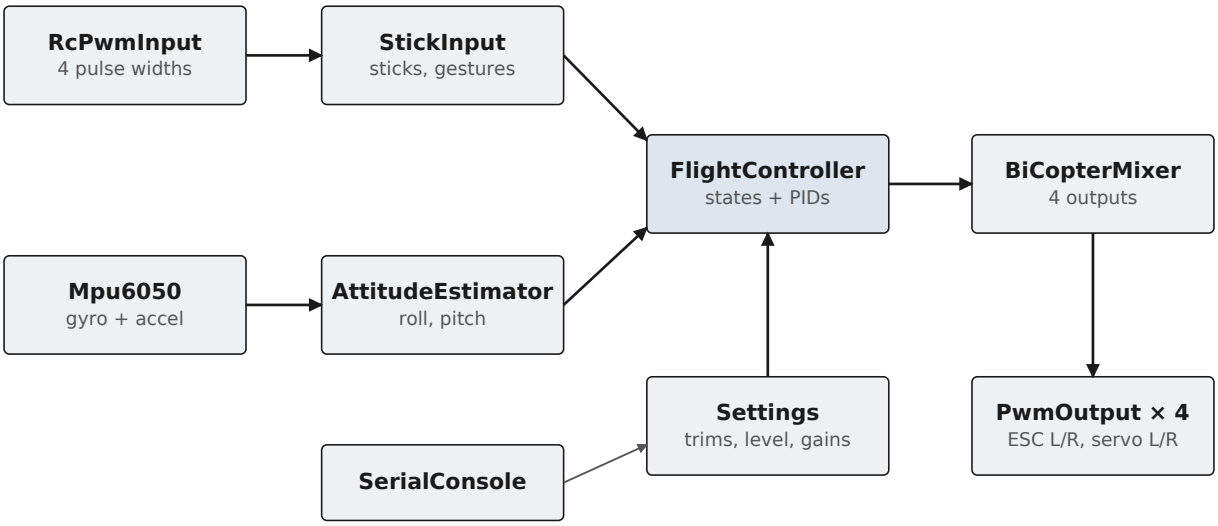


Figure 4. Data flow through the libraries in one pass of the control loop.

Table 3. Firmware libraries.

Library	Function
Mpu6050	Burst read of gyroscope, accelerometer and temperature; gyroscope calibration at start-up; mapping of the sensor axes to the body frame.
RcPwmInput	Measurement of the four receiver pulses with the MCPWM capture hardware, independent of interrupt latency; median-of-three glitch filter.
StickInput	Scaling of the sticks, learning of the stick centres, receiver failsafe detection, arming and calibration gestures.
AttitudeEstimator	Mahony filter with smooth accelerometer weighting and gyroscope bias learning.
Pid, Pt1Filter, BiquadFilter	Rate controller and the low-pass filters.
LeadCompensator	Optional phase lead for the servo commands (disabled by default).
BiCopterMixer, PwmOutput	Mixing to four pulse widths, generated by the LEDC peripheral at 400 Hz (ESCs) and 100 Hz (servos).
FlightController	State machine, safety logic and control laws.
Settings, SerialConsole, TuningWeb	Non-volatile parameters, USB console, WiFi tuning page and flight recorder.

5.2 Real-time loop

The control loop runs every 2 ms. Each pass reads the sensor, updates the attitude estimate, reads the sticks, runs the state machine and the controllers, and writes the outputs. Reading the sensor over I2C takes about 0.4 ms. The longest pass is reported continuously so that timing regressions are visible immediately.

5.3 Development and test tools

Two tools supported the development. A USB console streams the controller's internal state ten times per second and accepts commands for calibration and gain changes. A web page served by the ESP32 over its own WiFi network shows the attitude live, allows gains to be changed from a phone and records flights at 25 samples per second to a CSV file. Most design decisions in this report were taken after analysing such recordings.

6 Experimental Results

6.1 Test method

Tests were carried out with the aircraft held by hand, on a stand and on a tether. Two data sources were used: the console stream at 10 Hz and the web recorder at 25 Hz. Two results were obtained by simulation; they are labelled as such. All figures are generated by tools/make_figures.py from the data in docs/data and from the simulations.

6.2 Roll and pitch behaviour

Figure 5 shows 20 s of a logged throttle sweep with an earlier version of the controller. The roll axis, actuated by the motors, stays within 6 deg of level. The pitch axis, actuated by the servos, swings by up to 25 deg, and its command remains positive for long periods because the centre of gravity lay behind the rotor axis, so the servos had to hold the rotors tilted forward.

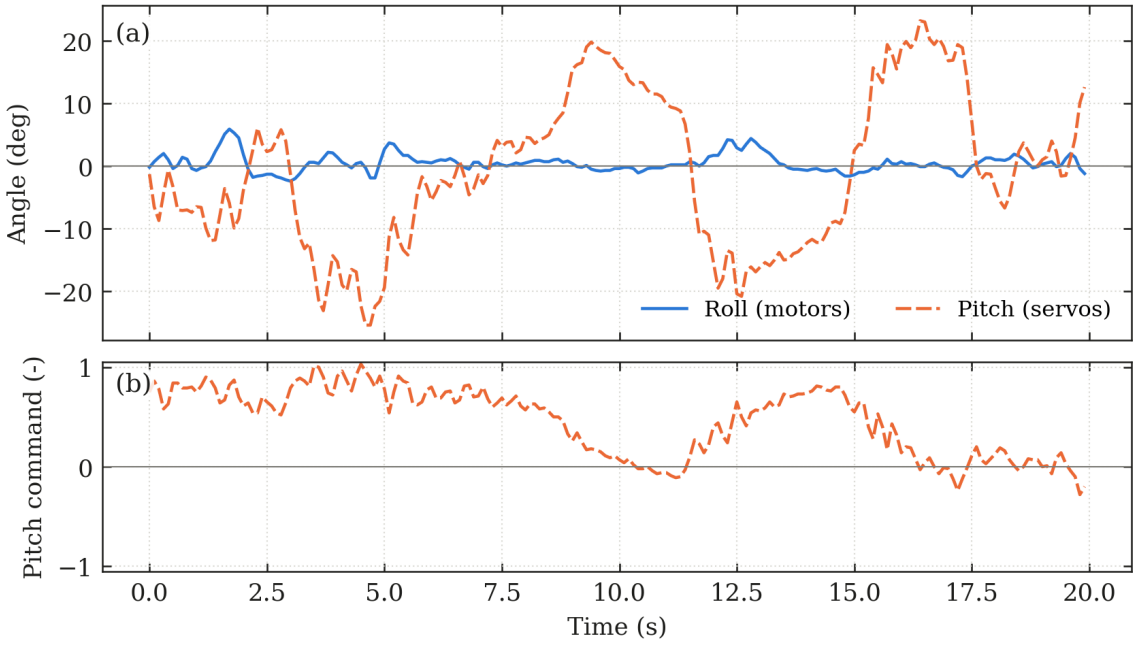


Figure 5. Measured (a) roll and pitch angle and (b) pitch servo command during a throttle sweep at 25 % throttle (console log, earlier controller version).

6.3 Attitude estimation

Figure 6 compares the complementary filter used initially with the Mahony filter of Section 4.2 on the same simulated hover, including a gyroscope bias, propeller vibration and the accelerations caused by servo corrections. The simulation calls the firmware's own estimator code. The complementary filter retains a constant error of about 1.5 deg, the bias multiplied by its time constant, while the Mahony filter learns the bias within about 30 s (RMS error 0.13 deg against 1.45 deg).

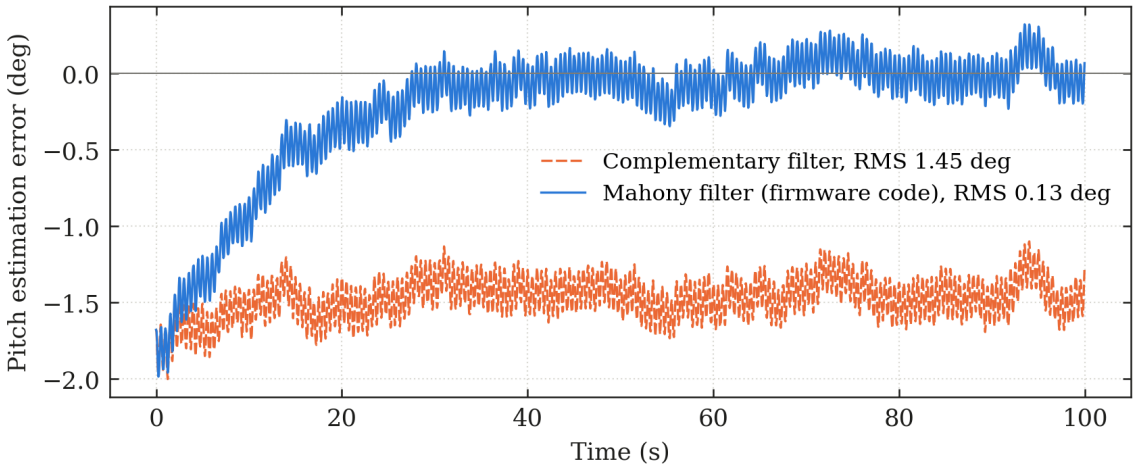


Figure 6. Simulated pitch estimation error of the complementary filter and the Mahony filter.

6.4 Servo response

Figure 7 shows the response of a simple MG996R-like servo model to a 10 deg step. A phase lead of 40 ms reduces the time to 90 % from 112 ms to 30 ms, at the cost of a small overshoot. On the real aircraft, combined with a higher pitch gain, the lead caused vibration, possibly through the reaction torque of the motor mounts when the servos accelerate; it is therefore disabled by default.

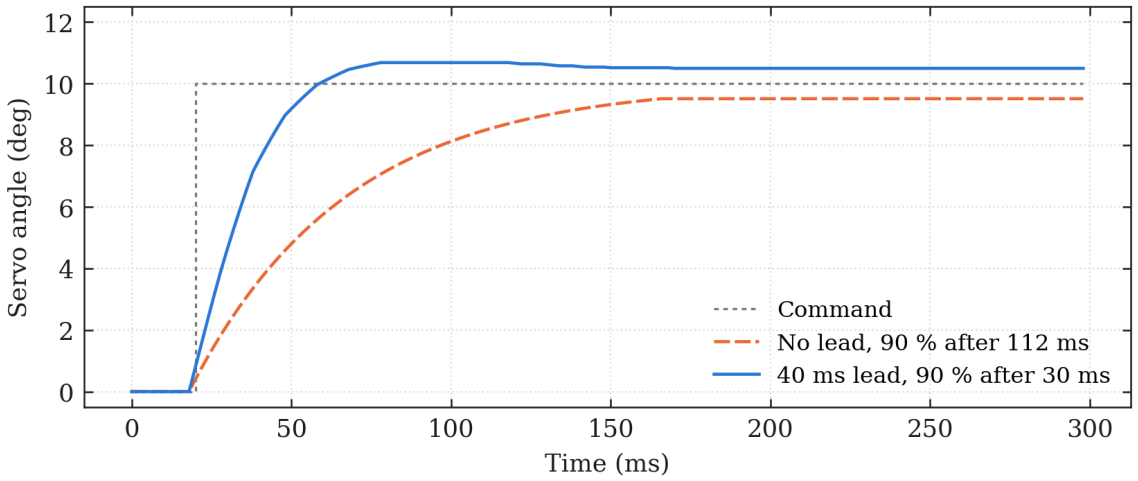


Figure 7. Simulated servo step response with and without a 40 ms phase lead (model, not a measurement).

6.5 Recorded flight

A 22 s flight was recorded on 23 October 2026 with the web recorder, of which 20 s were airborne at 63 % throttle. The complete record is available as docs/data/bicopter_20261023_182347.csv. It was flown with an earlier firmware version in which a hover trim was active, so the pitch target was about 9 deg nose-up. Table 4 summarises the airborne part.

Table 4. Statistics of the recorded flight (airborne part, throttle above 30 %).

Quantity	Value
Airborne time and throttle	20.2 s at 63 %
Roll angle, mean and standard deviation	+1.1 deg, 5.3 deg
Pitch angle, mean and standard deviation	−18.6 deg, 14.1 deg
RMS rate tracking error: roll / pitch / yaw	16 / 80 / 22 deg/s
Mean ESC pulse, left minus right	+90 µs
Motor tilt learned in hover	+9.3 deg (forward)
Samples in which the accelerometer was trusted	44 %

Figure 8 shows the attitude over the whole flight. Roll remains close to level apart from short excursions, whereas pitch repeatedly drops 40-55 deg nose-up and recovers, with a period of about 2.5 s.

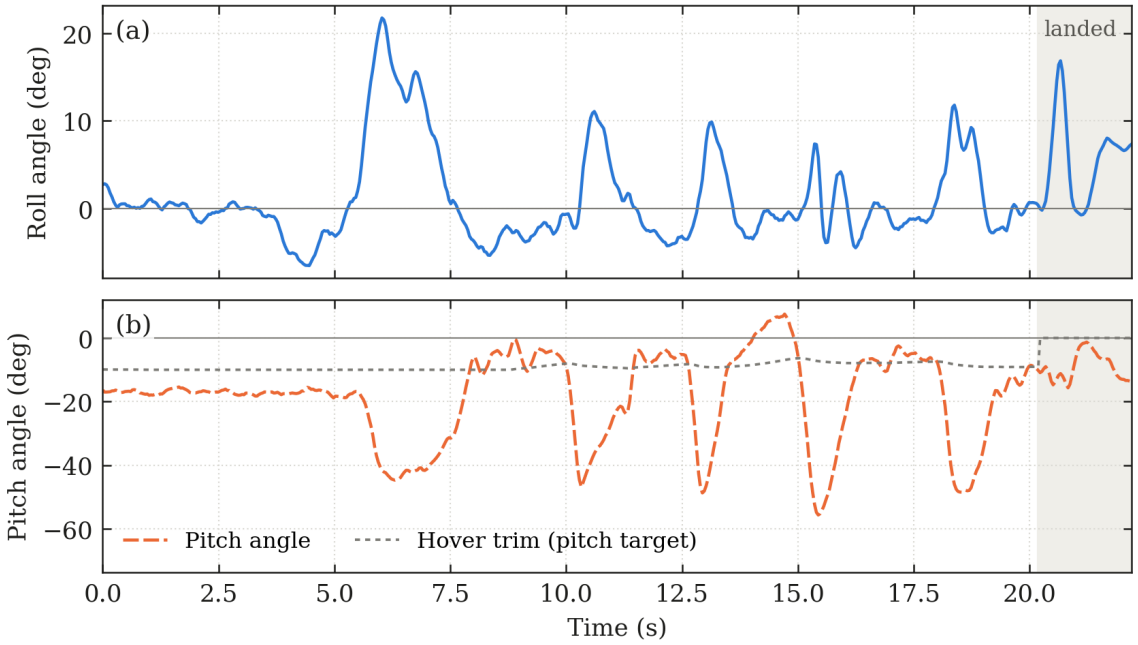


Figure 8. Recorded flight: (a) roll angle; (b) pitch angle and its hover-trim target.

Figure 9 compares commanded and measured rates on a common scale. On roll the two follow each other closely. On pitch the commanded rate saturates at its 90 deg/s limit for almost two seconds while the aircraft barely responds, after which the measured rate oscillates around the command: the servos cannot generate the pitch moment fast enough.

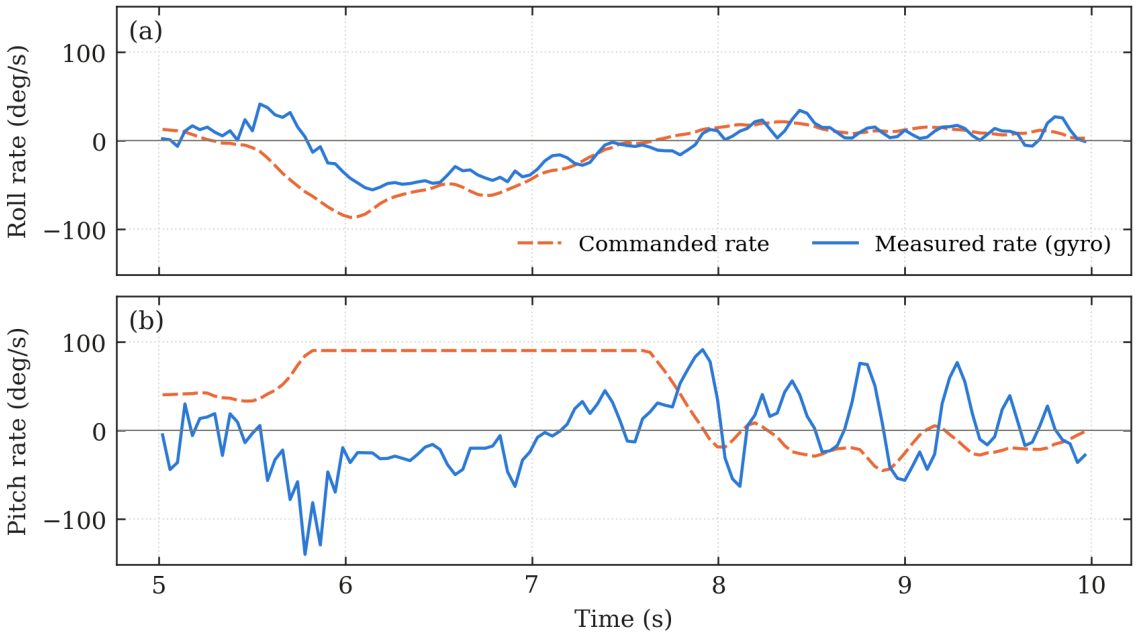


Figure 9. Commanded and measured angular rate for (a) roll and (b) pitch, 5 s of the recorded flight.

Figure 10 shows the actuator outputs. The left motor runs about 90 µs above the right one for most of the flight, indicating either a centre of gravity offset to the left or less thrust from the left rotor. Both servos remain away from their 1500 µs centre, holding the rotors tilted forward, which is consistent with a centre of gravity behind the rotor axis and with the learned hover tilt of +9.3 deg.

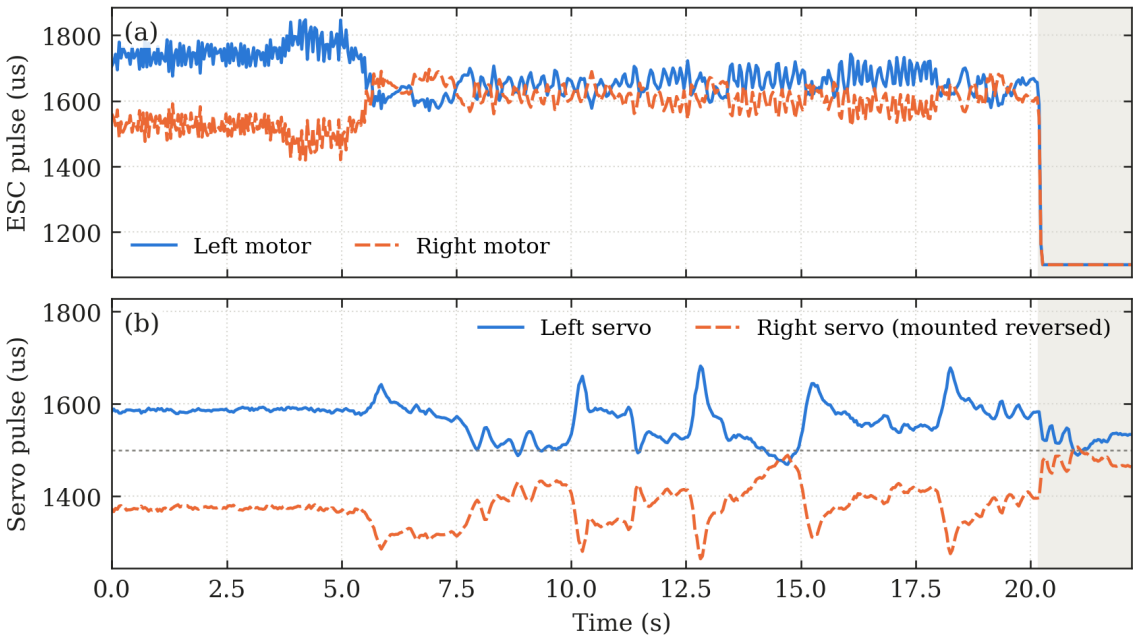


Figure 10. Recorded flight: (a) ESC pulses; (b) servo pulses. The right servo is mounted mirrored and moves in the opposite direction for the same tilt.

Figure 11 condenses the comparison: the servo-actuated pitch axis tracks its commanded rate five times worse than the motor-actuated roll axis.

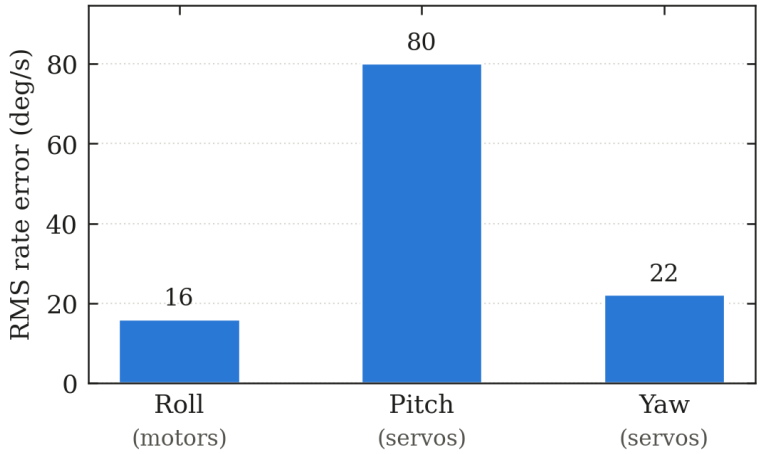


Figure 11. RMS rate tracking error per axis over the airborne part of the recorded flight.

7 Discussion

The results point to a single dominant cause: on this tilt-rotor bicopter the pitch axis is limited by its mechanics rather than by the controller. The control moment of the pitch axis is the thrust multiplied by the tilt angle and by the height of the rotors above the centre of gravity, a far shorter lever than the arm through which roll acts. The MG996R servos add roughly 100 ms of response time and some free play, and every pitch correction also produces a horizontal force that moves the aircraft forward or back. Where the centre of gravity lies behind the rotor axis, the servos must additionally hold a constant tilt, which consumes the travel needed for corrections and drives the pitch command into saturation.

Several controller-induced contributions were identified and removed: integrator wind-up on the ground, a stick offset integrated by the yaw loop, an accelerometer bias that caused the estimator to reject the accelerometer, and additional layers on the pitch controller that amplified fast servo movements. After these corrections the remaining behaviour is consistent with the mechanical explanation above.

7.1 Limitations

- The controller stabilises attitude and heading but not position; without an optical-flow or position sensor the aircraft drifts slowly with air currents and small level errors.
- The tilt servos limit the achievable pitch and yaw bandwidth.
- During testing the aircraft required a high hover throttle, which leaves little thrust margin for corrections.
- Without a magnetometer the heading reference drifts with the gyroscope bias.
- The recorded flight data were obtained with an earlier firmware version; the effect of the final configuration has not yet been quantified in the same way.

8 Conclusions

A flight controller for a tilt-rotor bicopter was designed, implemented on an ESP32-S3 and evaluated with logged flight data. The roll axis, actuated by differential thrust, proved easy to stabilise. The pitch axis, actuated by servos, tracked its commanded rate five times worse, and the analysis showed this to be mainly a mechanical limitation. A Mahony estimator with bias learning, take-off detection and a simplified pitch controller removed the contributions of the firmware itself.

References

[1] R. Mahony, T. Hamel and J.-M. Pflimlin, "Nonlinear complementary filters on the special orthogonal group," *IEEE Transactions on Automatic Control*, vol. 53, no. 5, 2008.

[2] S. O. H. Madgwick, "An efficient orientation filter for inertial and inertial/magnetic sensor arrays," technical report, University of Bristol, 2010.

[3] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, 2008.

[4] Q. Quan, *Introduction to Multicopter Design and Control*. Springer, 2017.

[5] Y. Qin, W. Xu, A. Lee and F. Zhang, "Gemini: A compact yet efficient bi-copter UAV for indoor applications," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, 2020.

[6] ArduPilot, "Tilt rotor planes (bicopter frame class)." https://ardupilot.org/plane/docs/guide-tilt-rotor.html

[7] P. Cook, "PlainFlightController." https://github.com/plainFlight/plainFlightController

[8] N. Rehm, "dRehmFlight VTOL flight controller." https://github.com/nickrehm/dRehmFlight

[9] R. Lopez, "ESP-FC: a flight controller for ESP32." https://github.com/rtlopez/esp-fc

[10] Espressif Systems, "ESP-Drone." https://github.com/espressif/esp-drone

[11] Betaflight, "Betaflight documentation: PID tuning and filtering." https://betaflight.com/docs

[12] Espressif Systems, *ESP32-S3 Technical Reference Manual*, chapters on MCPWM and the LED PWM controller.

[13] Espressif Systems, "Arduino-ESP32 documentation." https://docs.espressif.com/projects/arduino-esp32

[14] InvenSense (TDK), *MPU-6000 and MPU-6050 Product Specification* (PS-MPU-6000A-00) and *Register Map and Descriptions* (RM-MPU-6000A-00).

[15] PowerFeather, "ESP32-S3 PowerFeather documentation." https://powerfeather.dev

[16] PlatformIO, "PlatformIO documentation." https://docs.platformio.org

[17] FlySky, *FS-i6 Digital Proportional Radio Control System: User Manual*.

Appendix A Hardware Wiring

Table 5. Pin assignment of the ESP32-S3 PowerFeather.

Signal	GPIO	Board label	Connected to
MPU6050 SDA / SCL	35 / 36	SDA / SCL	GY-521 SDA / SCL
Receiver CH1 (roll)	39	SCK	receiver CH1 signal
Receiver CH2 (pitch)	40	MO	receiver CH2 signal
Receiver CH3 (throttle)	41	MI	receiver CH3 signal
Receiver CH4 (yaw)	42	RX	receiver CH4 signal
Left / right tilt servo	15 / 16	D5 / D6	servo signal
Left / right ESC	6 / 9	D8 / A1	ESC signal
Status LED	46	on board	-

The servos and the receiver are supplied by a separate 5-6 V regulator rated for at least 5 A, with a 470-1000 µF capacitor across the servo supply. All grounds are connected. The firmware enables the 3V3 rail (GPIO 4), which powers the inertial sensor, during start-up. The sensor is mounted near the centre of the frame on soft foam, with its X axis towards the nose and its Y axis towards the left rotor; other orientations are configured in include/Config.hpp.

Appendix B Setup and Tuning Procedure

All steps up to the first hover are carried out with the propellers removed. Each step verifies one item, so that a fault is found where it occurs. The detailed procedure is maintained in docs/SETUP_AND_TUNING.md; the essential steps are:

- **Build and flash** with PlatformIO (`pio run -e powerfeather -t upload`) and open the console (`pio device monitor`). Keep the aircraft still while the gyroscope is calibrated and the stick centres are learned.
- **Radio.** Roll right, pitch forward and yaw right must read positive. Set the receiver failsafe on the transmitter so that a lost link produces an out-of-range throttle value, and verify it by switching the transmitter off.
- **Sensor direction and level.** Right side down must give positive roll and nose down positive pitch; then store the level attitude on a flat surface.
- **Servos.** Both rotors vertical at centre (trims). Tilting the nose down by hand must tilt both rotors back; turning the nose left must tilt the left rotor forward.
- **Motors.** Calibrate the ESCs once, verify opposite propeller directions and the sign of the roll correction.
- **First hover and tuning.** Tune one axis at a time: increase P until the axis begins to oscillate and reduce it to about 70 %; increase I until slow drift disappears; use D on roll only.

The pitch axis is started from a plain PID (rate P 0.0030, I 0.0015, D 0, angle P 2.5). Gains can be changed live from the console or the web page and are stored with the save command.

Appendix C Console Commands

Table 6. USB console commands. Commands that change parameters are accepted only when disarmed.

Command	Function
h	help
s	status line on or off (once per second)
i / r / o / f	stream IMU, radio, outputs or the full flight log (10 Hz)
x	stop all output
g	recalibrate the gyroscope (aircraft still)
l	store the current attitude as level
trim l|r N	servo centre trim in microseconds
pid, pid r|p|y P I D	show or set the rate gains
ang r|p|y P	self-levelling gain (roll, pitch) or heading-hold gain (yaw)
lead S	servo phase lead in seconds, 0 = off (until reboot)
save / defaults	store parameters / restore the configured defaults
reboot	restart the processor

Appendix D Problems Encountered During Testing

The following problems occurred while the firmware was brought up on the aircraft. Most of them first appeared to be tuning problems.

Table 7. Problems encountered, their causes and the corrective measures.

Symptom	Cause	Measure
Console output almost absent	USB transmit buffer of 64 bytes dropped longer lines.	Output queued in a 2 KB buffer.
Attitude drifted, never settled	Accelerometer read 1.42 g at rest and was rejected.	Z offset measured at each start-up and removed.
Roll and pitch swapped	Sensor mounted rotated by 90 deg.	Axis mapping corrected; mirror images rejected at compile time.
Continuous yaw rotation	Yaw stick rested at 1438 µs due to a transmitter trim.	Stick centres learned at start-up; wider yaw deadband.
Servos slowly diverged	Throttle movement nudged the yaw stick; yaw integrator wound up.	Wider yaw deadband; yaw integrator shown in the flight log.
Both servos crept forward	Level attitude never calibrated (−8 deg offset).	Level calibration.
Fault state at start-up	Aircraft moved during gyroscope calibration.	Calibration repeats until the aircraft is still.
First upload always failed	USB port re-enumerates when entering the bootloader.	1200-baud reset before upload.
Lunge forward at take-off	Pitch integrator wound up against the ground.	Take-off detection (Section 4.5).
Forward/back oscillation	Slow servos; controller layers amplified fast servo starts.	Plain pitch PID; low and balanced centre of gravity.